BÖLÜM 4 / MODULE 4

Eser Zinciri (Artifact Chain) & PlantUML Mimarisi

The Artifact Chain & PlantUML Architecture

AI Unified Process'in 6 halkalı eser zinciri: Vizyondan koda tasarım uzayının daraltılması, Göbek ve Parmaklıklar (Hub-and-Spokes) modeli, PlantUML kullanım senaryosu diyagramı, <<include>> ve <<extend>> tuzakları, mekanik arayüz olarak kararlı kimlikler (FR, UC, BR, TC) ve /spec-review otomatik denetim mimarisi.

The 6-link AIUP artifact chain narrowing design space from vision to code, the Hub-and-Spokes architecture, PlantUML diagram as a table of contents, the truth about <<include>> and <<extend>>, identifiers as mechanical interfaces, and the two-pass /spec-review engine.

19

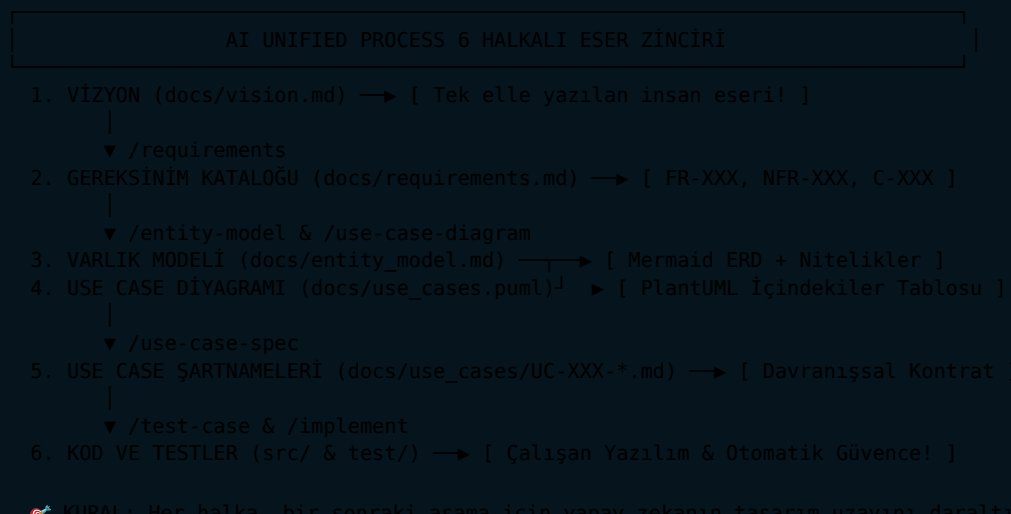
BÖLÜM: ESER ZINCIRI (ARTIFACT CHAIN) & PLANTUML MIMARISI | MODULE: THE ARTIFACT CHAIN & PLANTUML ARCHITECTURE

6 Halkalı Eser Zinciri: Vizyondan Koda Tasarım Uzayını Daraltma

The 6-Link Artifact Chain: Narrowing Design Space from Vision to Code

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir roket fırlatırken hedefe ulaşmak için 6 aşamalı bir kilit sistemi kullanırsın: 1) Haritada Mars'ı gösterirsin (Vizyon). 2) İhtiyaçları yazarsın: 'Oksijen, yakıt, 3 astronot' (Gereksinimler). 3) Roketin parçalarını çizersin (Varlık Modeli). 4) Görev adımlarını listelersin (Diyagram). 5) Her düğmenin ne zaman basılacağını kâğıda dökersin (Use Case). 6) Motorları ateşlersin (Kod!). Eğer direkt 1. adımdan 6. adıma atlarsan roket havada patlar! Zincirdeki her halka roketin rotasını milimetrik daraltır.

🇬🇧 ENGLISH (ELI5)

Launching an interplanetary rocket requires a six-stage telemetry sequence: 1) Pointing at Mars on the celestial map (Vision). 2) Sizing fuel and payload specifications (Requirements Catalog). 3) Blueprinting module geometries (Entity Model). 4) Indexing operational phases (Diagram). 5) Authoring step-by-step launch protocols (Use Case Specs). 6) Igniting the thrusters (Code & Tests). Skipping directly from Vision to Ignition ensures catastrophic atmospheric explosion. Each artifact progressively restricts variance until implementation becomes deterministic.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Eser Zinciri (The Artifact Chain) ve Tasarım Uzayı Mekaniği: Bir LLM'e tek satırlık bir prompt verip 'Bana bir e-ticaret sistemi yaz' dediğinizde tasarım uzayı (design space) sonsuzdur; model milyonlarca farklı mimari, tablo ve API rotası seçebilir ve bu da determinizmi sıfırlar. AI Unified Process bu sorunu 'Aşamalı Kısıtlama Zinciri' ile çözer: 1) docs/vision.md: Projede tek elle yazılan insan eseridir. Sistemin sınırlarını, aktörlerini ve alan adlarını belirler. 2) docs/requirements.md: Niyeti FR (Kullanıcı Hikayeleri), NFR ve Kısıtlar (Constraints) olarak kodlar. 3) docs/entity_model.md: Domain terimlerini ve ilişkilerini tek bir yerde tanımlar. 4) docs/use_cases.puml: Sistemin tüm use case haritasını çizer. 5) docs/use_cases/UC-XXX.md: Davranışı adım adım kilitler. `/implement` komutu çalıştığında yapay zekanın elinde vizyon, veri modeli, gereksinim satırı ve use case zaten mevcuttur; bu sayede üretilen kod tesadüfi değil, matematiksel bir kesinlik taşır. Hiçbir tekil dökümanın kusursuz olması gerekmez; bu katmanlı yapı sistemi sağlam kılar.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The mechanics of Design Space Constriction across the Artifact Chain: Prompting an LLM with 'Generate an e-commerce platform' presents an unconstrained, near-infinite degrees-of-freedom search space. The resulting architecture is chaotic, unprincipled, and un-reproducible. The AI Unified Process resolves this via a deterministic pipeline: each artifact progressively constricts the degrees of freedom for the next command. The vision document anchors boundary scope; the requirements catalog maps user stories to alphanumeric IDs; the entity model binds database nouns; the PlantUML diagram indexes the interaction portfolio; use case specifications define the step contracts. When `/implement` executes, the AI operates inside a tightly fenced sandbox. Reliable generation emerges from structural layering, not magical prompt engineering.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Eser Zincirinin Tamamını Yansıtan Temiz Mimari Yapısı
package com.maang.aiup.chain;

public class ArtifactChainDemonstration {
    // 1. docs/entity_model.md'den türeyen Domain Modeli
    public record BookCopyEntity(String copyId, String isbn, boolean isAvailable) {}

    // 2. docs/use_cases/UC-002-borrow-book.md'den türeyen Use Case Arayüzü
    public interface BorrowBookPort {
        void execute(String memberId, String copyId);
    }

    // 3. /implement tarafından üretilen temiz servis uygulaması
    public static class BorrowBookServiceImpl implements BorrowBookPort {
        @Override
        public void execute(String memberId, String copyId) {
            System.out.printf("Zincir tamamlandı: Üye %s -> Kopya %s başarıyla bağlandı.%n", memberId,
copyId);
        }
    }

    void main() {
        var service = new BorrowBookServiceImpl();
        service.execute("MEM-101", "CPY-99");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Zincirdeki halkaları atlayıp doğrudan /use-case-spec veya /implement çalıştırmaya kalkmayın! Elinde bir varlık modeli (entity model) olmayan AI, her use case'de aynı tablo için farklı kolon isimleri uydurur (örn: bir yerde memberId, diğerinde user_id) ve veritabanı kilitlenir.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never short-circuit the artifact chain by executing `/implement` without an entity model. Deprived of a canonical entity dictionary, the AI will invent conflicting field names across specifications (e.g. `memberId` vs `user_id` vs `account_num`), triggering schema drift.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Eser zinciri 6 halkadan oluşur: Vizyon, Gereksinimler, Varlık Modeli, Diyagram, Şartnameler, Kod/Test.
- Tek elle yazılan döküman vizyondur; diğer tüm eserler komutlarla AI tarafından taslaklanır ve insan tarafından onaylanır.
- Her halka bir sonrakinin tasarım uzayını daraltarak rastgele kod üretimini deterministik kılar.

🎯 Key Architectural Takeaways

- The artifact chain spans 6 links: Vision, Requirements, Entity Model, Diagram, Specs, Code/Tests.
- Vision is the sole hand-written artifact; downstream artifacts are generated and human-refined.
- Each artifact narrows the design space, making AI code synthesis deterministic and predictable.

20

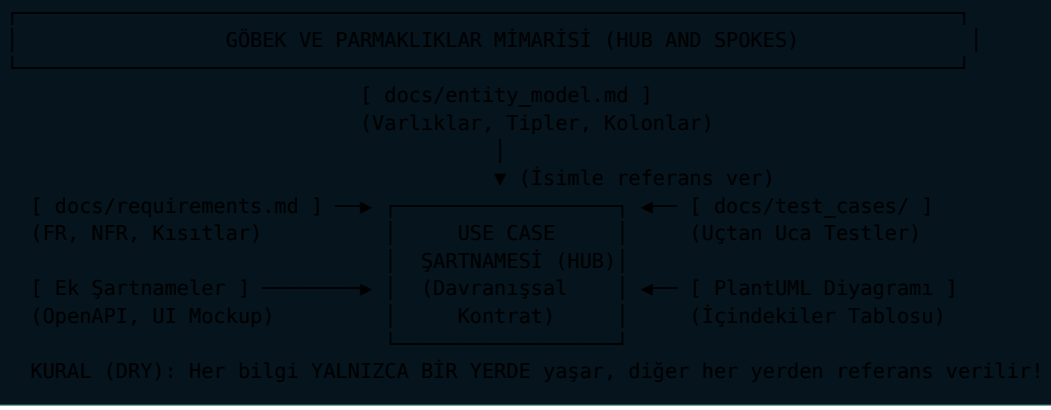
BÖLÜM: ESER ZINCIRI (ARTIFACT CHAIN) & PLANTUML MIMARISI | MODULE: THE ARTIFACT CHAIN & PLANTUML ARCHITECTURE

Göbek ve Parmaklıklar Modeli (Hub & Spokes): Use Case Neden Sadece 'Bölüm 3'tür?

The Hub-and-Spokes Model: Why Use Cases Are Chapter 3 of the Requirements

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir bisiklet tekerleğini düşünün: Ortada sağlam bir tekerlek göbeği (Hub) vardır ve bu göbekten dışarıya doğru onlarca metal parmaklık (Spokes) uzanır. Use Case bu tekerleğin tam merkezidir; ama tek başına gidemez! Bir parmaklık sözlüğe (Entity Model), diğer parmaklık hız sınırına (NFR), öbürü testlere gider. Eğer göbeğin içine lastiği, freni ve pedalları tıkıştırmaya çalışırsan tekerlek dönmez! Use case sadece 'kim ne yaptı' hikayesini anlatır; teknik detaylar parmaklıkların ucundaki kendi evlerinde yaşar.

🇬🇧 ENGLISH (ELI5)

Visualize a bicycle wheel: at the center sits the structural hub, with metallic spokes radiating outward to support the rim. The use case specification is the central hub. It governs behavioral flow, but delegates non-behavioral concerns out along its spokes: data definitions point to the Entity Model, performance budgets point to the NFR table, and user journeys point to Test Cases. Jamming database tables and UI wireframes directly into the hub fractures the wheel.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Cockburn'ün 'Bölüm 3' Metaforu ve DRY Prensibi: Cockburn, kullanım senaryolarının gereksinim dökümanının tamamı olmadığını, sadece 'Bölüm 3: Davranışsal Gereksinimler' olduğunu söyler. Diğer tüm konular use case'in dışındaki özel dosyalarda yaşar: 1) Veri Yapısı ve Alan Tipleri: `docs/entity_model.md` içinde tanımlanır; use case sadece 'Üye' veya 'Ödünç' ismini kullanır, asla içinde tablo şeması veya kolon listesi barındırmaz. 2) NFR ve Kısıtlar: `docs/requirements.md` içindeki NFR tablosunda yaşar; use case sadece `Requirements: [FR-012, NFR-002]` olarak linkler. 3) UI Mockup'ları ve OpenAPI: Ek dökümanlar olarak bağlanır. 4) Bilgi Tekrarı Yasağı (DRY): İnsan geliştiriciler için çift yazım sadece bir bakım zorluğuydu. Ancak yapay zekâ için çift yazım ÖLÜMCÜLDÜR! İki döküman zamanla birbirinden farklılaşır ve aralarında çelişki doğar. Bir AI ajanı çelişki gördüğünde bunu bir soru olarak sormaz; kafasına göre bir tarafı seçerek kod üretir. Bu yüzden her bilgi tam olarak TEK bir yerde tanımlanır ve sadece referans verilir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Cockburn's 'Chapter 3' Doctrine and Requirements-Level DRY: Cockburn famously noted that use cases constitute merely Chapter 3 of a complete requirements document: the behavioral slice. Everything that resists the narrative structure is spun off into dedicated satellites. In the AI Unified Process, these spokes are mechanically formalized: 1) Data Schemas live in `docs/entity_model.md`. The spec cites domain nouns by business name and never inlines attribute tables. 2) Non-functional constraints reside in `docs/requirements.md` and are referenced via metadata tags. 3) Contract interfaces (OpenAPI, UI wireframes) are hyperlinked supplementary files. 4) The AI-Era DRY Imperative: For human teams, duplicated requirements created synchronization debt. For LLMs, duplication produces catastrophic failure: when two documents diverge by a single field, the AI resolves the contradiction silently, picking an arbitrary side. DRY is no longer merely a coding aesthetic; it is an existential requirements invariant.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Hub & Spokes Prensibine Göre Ayrıştırılmış Temiz Domain Mimarisi
package com.maang.aiup.hubspokes;

// SPOKE 1: Domain Modeli (docs/entity_model.md karşılığı)
record Member(String id, String name, String email) {}
record BookCopy(String copyId, String isbn) {}

// SPOKE 2: İş Kuralı Portu (docs/requirements.md karşılığı)
interface LoanPolicyGate {
    boolean canBorrow(Member member);
}

// HUB: Use Case İcra Sınıfı (docs/use_cases/UC-002 karşılığı - Sadece Akışı Yönetir!)
public class BorrowBookHubUseCase {
    private final LoanPolicyGate policyGate;

    public BorrowBookHubUseCase(LoanPolicyGate policyGate) {
        this.policyGate = policyGate;
    }

    public void execute(Member member, BookCopy copy) {
        // Hub sadece orkestrasyon yapar, veritabanı veya karmaşık kural detaylarını spokelara delege
eder
        if (!policyGate.canBorrow(member)) {
            throw new IllegalStateException("Ödünç alma kuralı ihlal edildi!");
        }
        System.out.printf("Hub tamamlandı: %s üyesine %s kitabı tahsis edildi.%n", member.name(),
copy.isbn());
    }

    void main() {
        var hub = new BorrowBookHubUseCase(_ -> true);
        hub.execute(new Member("M1", "Ada Lovelace", "ada@lovelace.org"), new BookCopy("C1",
"978-01"));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Use case şartnamesinin içine SQL 'CREATE TABLE' komutları veya ekranın piksel genişliklerini ASLA yazmayın! Bu detaylar parmaklıkların ucundaki varlık modeline ve UI mockup dosyalarına aittir. Göbeği şişirmek AI'ın aklını karıştırır.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never embed DDL statements (`CREATE TABLE`), column data types, or pixel coordinates directly inside a use case scenario. Inlining schema definitions bloats the hub and invites silent synchronization drift.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Use Case tekerleğin göbeğidir; sadece davranışsal akışı yönetir, detayları spokelara devreder.

• Veri yapıları entity_model.md'de, performans NFR tablosunda, testler ise test_cases klasöründe yaşar.

• DRY artık sadece kodlama değil, şartname kuralıdır: Çelişkili iki bilgi AI'ın sessizce yanlış kod yazmasına yol açar.

🎯 Key Architectural Takeaways

• The use case is the structural hub; it orchestrates behavior while delegating details to spokes.

• Data schemas live in the entity model, NFRs in the catalog, and journeys in test cases.

• DRY is a requirements imperative: contradictions across duplicated specs cause silent AI bugs.

21

BÖLÜM: ESER ZINCIRI (ARTIFACT CHAIN) & PLANTUML MIMARISI | MODULE: THE ARTIFACT CHAIN & PLANTUML ARCHITECTURE

PlantUML Use Case Diyagramı: Sistemin İçindekiler Tablosu (Modelin Kendisi Değil!)

The PlantUML Use Case Diagram: Table of Contents, Not the Model

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
                PLANTUML DİYAGRAMI VE VERBATIM DOSYA EŞLEŞMESİ

PlantUML Kodu (docs/use_cases.puml):
@startuml
actor "Member" as member
rectangle "Library System" {
  usecase "UC-001\nSearch Catalog" as UC001
  usecase "UC-002\nBorrow Book" as UC002
}
member --> UC001
member --> UC002
@enduml

BİREBİR DOSYA SİSTEMİ EŞLEŞMESİ (VERBATIM FILE SYSTEM):
docs/use_cases/
├── UC-001-search-catalog.md  ◄── (UC-001\nSearch Catalog ile noktası virgülüne aynı!)
└── UC-002-borrow-book.md     ◄── (UC-002\nBorrow Book ile noktası virgülüne aynı!)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kitaptaki 'İçindekiler' sayfasını düşünün: Sayfada sadece 'Bölüm 1: Giriş - Sayfa 15' yazar. İçindekiler sayfasına oturup tüm romanı sığdırmaya çalışmazsın; sadece hangi konunun nerede olduğunu tek bir bakışta gösterir. PlantUML diyagramı da yazılımın İçindekiler tablosudur! Yuvarlakların içine tüm detayları yazamazsın; yuvarlak sadece kapıdaki tabeladır, asıl oda arkasındaki markdown dosyasındadır!

🇬🇧 ENGLISH (ELI5)

A book's Table of Contents does not contain the complete novel; it indexes chapter titles and page numbers so readers can navigate the landscape at a glance. A PlantUML use case diagram plays the exact same role. The ellipses are not the system architecture; they are visual signposts. Attempting to cram complex algorithms into diagram bubbles ruins both the diagram and the specification.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

PlantUML Diyagramının Mühendislik Sınırları ve Kuralları: Cockburn use case diyagramlarının modelin kendisi olmadığını, sadece bir 'Büyük Resim / İçindekiler Tablosu' olduğunu ısrarla vurgulamıştır. `docs/use_cases.puml` dosyası downstream çalışan AI araçları için bir haritadır. Bu haritanın kusursuz çalışması için Martinelli 4 katı kural koyar: 1) İki Satırlı Etiket: Her use case yuvarlağı ID ve İsmi iki satırda taşımalıdır: `UC-001\nSearch Catalog`. 2) Gerçek Sistem İsmi: `rectangle` etiketinde soyut isimler değil, projenin gerçek sistem adı yazmalıdır (`rectangle "Library System"`). 3) Her Aktör Bağlı Olmalıdır: Diyagramdaki her aktör en az bir use case'e, her use case de en az bir aktöre bağlanmalıdır; yetim yuvarlak kalamaz. 4) Verbatim Eşleşme: Diyagramdaki isim kebab-case yapılarak dosya adına dönüşür: `UC-002\nBorrow Book` -> `docs/use_cases/UC-002-borrow-book.md`. `/spec-review` komutu bu eşleşmeyi otomatik doğrular; uyuşmazlık varsa CI derlemesini kırar.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

PlantUML Diagram Semantics and Verification Protocol: Cockburn continually warned against treating use case diagrams as architectural models. In the AI Unified Process, `docs/use_cases.puml` serves strictly as the system's navigable index. Downstream parsing engines enforce four non-negotiable conventions: 1) Two-Line Labeling: Every use case node must declare its alphanumeric identifier and human name across two lines: `UC-001\nSearch Catalog`. 2) System Boundary: The enclosing bounding box must declare the authentic bounded-context system name. 3) Completeness Invariant: Every defined actor must link to at least one use case; isolated nodes fail verification. 4) Verbatim Filename Parity: The string literal in the diagram maps deterministically to the file system: `UC-002\nBorrow Book` maps exclusively to `docs/use_cases/UC-002-borrow-book.md`. Automated tools parse this contract to guarantee zero orphan specifications.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: PlantUML Eşleşmesini Otomatik Denetleyen Doğrulama Kodu
package com.maang.aiup.diagram;

import java.nio.file.Files;
import java.nio.file.Path;

public class DiagramParityChecker {
    // PlantUML dosyasındaki 'UC-002\nBorrow Book' tanımı:
    public record UseCaseEntry(String id, String rawName) {
        public String expectedFileName() {
            String sanitized = rawName.toLowerCase().replace(" ", "-");
            return id + "-" + sanitized + ".md";
        }
    }

    public static boolean verifyFileExists(Path baseDir, UseCaseEntry entry) {
        Path expectedPath = baseDir.resolve(entry.expectedFileName());
        boolean exists = Files.exists(expectedPath);
        System.out.printf("Denetim: %s -> %s (Mevcut: %s)%n", entry.rawName(),
expectedPath.getFileName(), exists);
        return exists;
    }

    void main() {
        var entry = new UseCaseEntry("UC-002", "Borrow Book");
        System.out.println("Beklenen dosya: " + entry.expectedFileName());
        assert entry.expectedFileName().equals("UC-002-borrow-book.md") : "İsim eşleşme hatası!";
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

PlantUML diyagramını oklarla, koşullarla ve if-else mantıklarıyla boğmayın! Diyagram bir akış şeması (flowchart) değildir. Adımların sırası diyagramda değil, use case şartnamesinin içindeki numaralı adımlarda yaşar.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not treat a use case diagram as an execution flowchart. Avoid drawing conditional branching arrows between ellipses. The chronological flow belongs exclusively within the numbered steps of the markdown specification.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- PlantUML diyagramı sistemin içindekiler tablosudur; modelin kendisi değil, haritasıdır.
- Diyagramdaki etiket iki satırdan oluşur: ID ve İsim (UC-001\nSearch Catalog).
- Diyagramdaki isim ile dosya adı (UC-001-search-catalog.md) noktası virgülüne eşleşmelidir.

🎯 Key Architectural Takeaways

- The PlantUML diagram is the system's table of contents, not the architectural model.
- Diagram nodes must declare ID and Name on two lines (e.g. UC-001\nSearch Catalog).
- The diagram name and the markdown filename must maintain 1:1 verbatim parity.

22

BÖLÜM: ESER ZINCIRI (ARTIFACT CHAIN) & PLANTUML MIMARISI | MODULE: THE ARTIFACT CHAIN & PLANTUML ARCHITECTURE

<<include>> ve <<extend>> Hakkındaki Gerçek: Neden Neredeyse Hiç İhtiyacınız Yok?

The Truth About <<include>> and <<extend>>: Why You Almost Never Need Them

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────────┐
│              <<INCLUDE>> VE <<EXTEND>> TUZAĞI VE MODERN ÇÖZÜM │
└──────────────────────────────────────────────────────────────┘

1. ESKİ YANLIŞ KULLANIM (AŞIRI MÜHENDİSLİK):
   [ Kitap Ödünç Al ] ──<<include>>──▶ [ Kullanıcıyı Giriş Yaptır ] (YANLIŞ!)
   [ Kitap Ödünç Al ] ──<<include>>──▶ [ Veritabanına Bağlan ]      (FACİA!)
   * Kimlik doğrulama bir include DEĞİLDİR; o bir ÖNKOŞULDUR (Precondition)!

2. MODERN AIUP KURALI:
   - <<include>>: SADECE gerçekten 3'ten fazla use case tarafından paylaşılan
                  ve bağımsız değeri olan ağır bir alt akış varsa kullanılır (Nadir!).
   - <<extend>>:  Yazılımı anlamayı imkansızlaştırır; AIUP'ta NEREDEYSE HİÇ KULLANILMAZ!
   - Kural: Basit tutun! Bir akış diğerini çağıracaksa referans verin: (UC-005 Reserve Book).
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski modelciler yemek yaparken her hareket için ayrı bir döküman açmaya bayılırdı: 'Yemek Pişir' use case'i, 'Tuzluğu Tut' use case'ini `\<\<include>>` eder; 'Tuzluğu Tut' ise 'Kapağı Aç'ı `\<\<include>>` ederdi. Sonunda kimse yemek pişiremez hale gelirdi! 'Giriş Yap' bir include değildir; mutfağa girmeden önce kapının anahtarla açılmış olması bir önkoşuldur (Precondition). Akıllı bir usta gibi davranın: Birbirine bağlı gereksiz oklar çizmeyi bırakın, sadece ana işi yazın!

🇬🇧 ENGLISH (ELI5)

Bureaucratic modelers loved fracturing simple recipes: 'Cook Omelet' would `\<\<include>>` 'Grasp Salt Shaker', which would `\<\<include>>` 'Unscrew Cap'. The kitchen ground to a halt under paper overload. Authentication is not an included sub-routine; having unlocked the kitchen door is a static Precondition. In the AI era, procedural spaghetti arrows between ellipses confuse AI parsers. Omit them.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

<<include>> ve <<extend>> İlişkilerinin Çöküşü: UML tarihindeki en büyük kafa karışıklığı ve zaman israfı bu iki anahtar kelimeden doğmuştur. Cockburn 2001'de açıkça uyarmıştı: 'Alt use case'ler pahalıdır; yalnızca yeniden kullanılabilirlik veya aşırı okunabilirlik gerektiren durumlarda mazur görülebilir. Pratikte bu kural şuna indirgenir: Neredeyse hiçbir zaman!' AI Unified Process'te bu kural daha da katılaşır: 1) Kimlik Doğrulama Bir Include Değildir: 'Kullanıcı Girişi' (Log In) asla bir include olarak her senaryoya bağlanamaz; o bir Önkoşuldur (Preconditions: The member is signed in). Önkoşul olarak yazıldığında bir rota korumasına (Route Guard) dönüşür ve akışın içinde tekrar tekrar doğrulanmaz. 2) <<extend>> Yasağı: Extend ilişkisi akışın dışarıdan bir şarta bağlı olarak use case'e enjekte edilmesidir; akışın nerede dallandığını gizler ve hem insanı hem AI'ı şaşırtır. AIUP'ta extend yerine Alternatif Akışlar (A1, A2) kullanılır. 3) Handoff by Reference: Eğer bir use case başka bir use case'i tetikleyecekse, ok çizmek yerine alternatif akışın içine parantez içinde link verilir: `3. If the member accepts, system records reservation (UC-005 Reserve Book).` Hafif, temiz ve sıfır karmaşıklık!

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Deconstruction of <<include>> and <<extend>>: UML literature generated decades of futile debate over relationship arrows. Cockburn's original rule of thumb: sub-use cases carry immense overhead; they are justified strictly by cross-functional reuse. In practice, this heuristic collapses to: almost never. In the AI Unified Process: 1) Authentication is never an Include: Injecting 'Log In' via `\<\<include>>` into 20 different use cases is an architectural anti-pattern. Authentication is a Precondition ('The member is signed in'). Preconditions translate cleanly into Spring Security filters or gateway route guards, bypassing in-flow redundant validations. 2) The Banishment of <<extend>>: Extension points fragment execution context across disjointed files, baffling human reviewers and confusing AI generators. All conditional deviations belong in named Alternative Flows (A1, A2). 3) Handoff by Reference: When a use case legitimately triggers another, it invokes it inline via alphanumeric citation: `(UC-005 Reserve Book)` without drawing complex UML dependency webs.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: <<include>> Yerine Önkoşul (Route Guard) ve Referans ile Çağrı Modeli
package com.maang.aiup.relationships;

public class RelationshipModernPatterns {
    // ❌ YANLIŞ: Her use case içinde kimlik doğrulama include'u çalıştırmak.
    // ✅ DOĞRU: Önkoşul (Precondition) güvenlik filtresinde çözülür:
    public record AuthenticatedUser(String id, String role) {}

    public static class CleanBorrowService {
        // Önkoşul sistem tarafından garantilenmiştir (Guarded Entry)
        public void borrow(AuthenticatedUser user, String bookId) {
            // Include yok! Doğrudan iş kuralı ve referans var:
            boolean bookAvailable = false;

            if (!bookAvailable) {
                // Handoff by reference: UC-005 Reserve Book tetiklenir!
                triggerReserveBookUseCase(user, bookId);
                return;
            }
            System.out.println("Ödünç tamamlandı.");
        }

        private void triggerReserveBookUseCase(AuthenticatedUser user, String bookId) {
            System.out.printf("UC-005 Reserve Book devreye girdi: %s için rezervasyon yapıldı.%n",
user.id());
        }
    }

    void main() {
        var service = new CleanBorrowService();
        service.borrow(new AuthenticatedUser("U1", "MEMBER"), "BK-44");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir PlantUML diyagramında 10'dan fazla <<include>> ve <<extend>> oku görüyorsanız o diyagram çöp olmuştur. Hepsini silin; kimlik doğrulamayı önkoşul yapın, dallanmaları ise alternatif akışlara taşıyın.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

If a use case diagram resembles a spiderweb of \<\<include>> and \<\<extend>> arrows, the model is deeply corrupted. Purge them. Move authentication to Preconditions and push deviations into local Alternative Flows.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Kimlik doğrulama bir <<include>> değildir; senaryo öncesi bir Önkoşuldur (Precondition).

• <<extend>> ilişkisi akışı gizler ve karmaşıklaştırır; AIUP'ta bunun yerine Alternatif Akışlar kullanılır.

• Başka bir use case'e geçiş, senaryo adımı içinde referans verilerek (UC-005 Reserve Book) hafifçe çözülür.

🎯 Key Architectural Takeaways

• Authentication is never an include; it is an established Precondition.

• `<<extend>>` obscures control flow; use explicit Alternative Flows instead.

• Hand off between use cases lightly by inline reference: (UC-005 Reserve Book).

23

BÖLÜM: ESER ZINCIRI (ARTIFACT CHAIN) & PLANTUML MIMARISI | MODULE: THE ARTIFACT CHAIN & PLANTUML ARCHITECTURE

Arayüz Olarak Kararlı Kimlikler: FR-001'den Koda Mekanik İzlenebilirlik Zinciri

Identifiers as Interfaces: Mechanical Traceability from FR-001 to Code

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────┐
│              MEKANİK İZLENEBİLİRLİK ZİNCİRİ (TRACEABILITY CHAIN)          │
└──────────────────────────────────────────────────────────┘

FR-005 (Kullanıcı Hikayesi)
   │
   ▼ realizes
UC-003 (Manage Books Use Case)
   │
   ▼ diverges at
A2 (Alternatif Akış: Stok Yetersiz)
   │
   ▼ enforces
UC-003 BR-007 (İş Kuralı: Asgari Rezervasyon Limiti)
   │
   ▼ generates
BookBorrowService.java (Kaynak Kod Satırı)
   │
   ▼ verifies
TC-003_A2_Test.java (Otomatik Test Senaryosu)

🎯 MÜKEMMEL DÜZEN: Bir müşteri "Bu sipariş niye reddedildi?" diye sorduğunda,
kimse kodun içinde kaybolmaz; kimlik zincirini (FR -> UC -> A2 -> BR) gösterir!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir otomobil fabrikasında her somunun ve her civatanın bir parça kodu (Örn: C-104) vardır. Araba arıza yaptığında tamirci 'Şu vidayı değiştir' demez; 'C-104 parçasını tak' der. Yazılımda da isimler değişkendir ama kimlikler (FR-001, UC-002, BR-004, TC-001) çelikten dökülmüş mühürlerdir! Bu mühürler sayesinde ilk müşteri talebinden en alttaki Java test koduna kadar her şey birbirine mekanik bir zincirle bağlanır.

🇬🇧 ENGLISH (ELI5)

In aeronautical engineering, every bolt and gasket carries a stamped serial code (e.g. PART-881). When an aircraft malfunctions, technicians don't say 'swap that little silver thing'; they reference the exact component code. In software, natural language drifts, but alphanumeric identifiers (FR-001, UC-002, BR-005, TC-003) are immutable interfaces. They bind client intent directly to automated tests with mechanical traceability.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Kararlı Kimlikler (Stable Identifiers) ve Mekanik Arayüz Mantığı: Geleneksel yazılım süreçlerinde isimlendirme kuralları (naming conventions) sadece bir 'temizlik ve düzen' (hygiene) meselesiydi. AI Unified Process'te ise bu kimlikler birer 'makine arayüzüdür' (interfaces). İnsanlar anlamı kelimelerden çıkarır; yapay zekâ araçları ise ilişkileri kimliklerle kurar. 1) Standart Kimlik Formatı: FR-XXX (Fonksiyonel Gereksinim), NFR-XXX (Fonksiyonel Olmayan), C-XXX (Kısıt), UC-XXX (Use Case), BR-XXX (İş Kuralı), TC-XXX (Test Senaryosu). 2) Yerel Numaralandırma ve Nitelikli Referans (Qualified Citation): BR numaralandırması her use case'in kendi içinde yereldir (`UC-001` dosyasında `BR-001..003`, `UC-002` dosyasında yine `BR-001..003` olabilir). Ancak bir use case dışarıdaki bir kurala atıfta bulunacaksa MUTLAKA nitelikli referans (qualified reference) vermek zorundadır: `UC-001 BR-002`. Başına use case kodu konulmayan çıplak bir `BR-002` sadece kendi dosyasının içinde geçerlidir. 3) Kesintisiz Zincir: Müşteri bir kuralı sorduğunda cevap kodun içinde aranmaz; `FR-005 -> UC-003 -> A2 -> UC-003 BR-007 -> Service.java -> Test.java` kimlik zinciri tek saniyede çıkarılır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Identifiers as Mechanical Interfaces: In legacy development, naming standards were mere aesthetic hygiene. In AIUP, identifiers are machine-parseable interfaces. While humans parse semantics from prose, LLM agent toolchains resolve relational graphs via alphanumeric coordinates. 1) Taxonomy: FR-XXX (Functional Requirement), NFR-XXX (Non-Functional), C-XXX (Constraint), UC-XXX (Use Case), BR-XXX (Business Rule), TC-XXX (Test Case). 2) Local Scope and Qualified Scoping: Business rule identifiers are local to their declaring use case (UC-001 has BR-001 through BR-003; UC-002 also starts at BR-001). Citing a rule across use case boundaries requires strict qualification: `UC-001 BR-002`. An unqualified `BR-002` is only valid within its host file. 3) Mechanical Audit Trails: When compliance audits require explaining an order rejection, the team provides an unbroken chain of identifiers: FR-005 -> UC-003 -> A2 -> UC-003 BR-007 -> code -> test. Nobody decompiles bytecode to discover business logic.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Kimlikleri Kod ve Test Açıklamalarında Taşıyan İzlenebilirlik
package com.maang.aiup.traceability;

/**
 * Traces to:
 * - Requirement: FR-012 (Borrow Book Flow)
 * - Use Case: UC-002 (Borrow Book)
 * - Flow: A2 (Loan Limit Reached)
 * - Rule: UC-002 BR-005 (Max 5 open loans)
 */
public class LoanLimitEnforcer {
    public static final String TRACE_RULE = "UC-002 BR-005";
    public static final int MAX_LOANS = 5;

    public static void verifyLimit(int currentLoans) {
        if (currentLoans >= MAX_LOANS) {
            // İstisna mesajı doğrudan kural kimliğini taşır!
            throw new IllegalStateException(TRACE_RULE + ": Maksimum ödünç limitine ulaşıldı.");
        }
    }

    void main() {
        try {
            verifyLimit(5);
        } catch (IllegalStateException e) {
            System.out.println("✅ Mekanik hata yakalandı: " + e.getMessage());
        }
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

İş kurallarının metnini başka bir use case'in içine kopyalayıp yapıştırmak KESİNLİKLE YASAKTIR! Kural kopyalanırsa zamanla biri güncellenir diğeri unutulur (drift). Kuralı daima ait olduğu use case'den kimliğiyle çağırın: 'UC-002 BR-004'.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Copying and pasting business rule text across specifications is strictly forbidden. Duplicated rules inevitably drift. Always cite external rules using their qualified identifier: `UC-002 BR-004`.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Kimlikler (FR, UC, BR, TC) birer hijyen değil, makinenin sistemi bağladığı arayüzlerdir.
- BR-XXX numaraları use case içinde yereldir; dışarıdan çağrılırken nitelikli yazılır (UC-001 BR-002).
- İzlenebilirlik zinciri sayesinde bir iş kuralı şartnameden koda ve teste kadar tek bakışta izlenir.

🎯 Key Architectural Takeaways

- Identifiers are not stylistic hygiene; they are mechanical interfaces connecting the chain.
- Business rule IDs are local to a spec; cross-spec citations must be qualified (UC-001 BR-002).
- The traceability chain allows instantaneous traversal from requirement to code and tests.

24

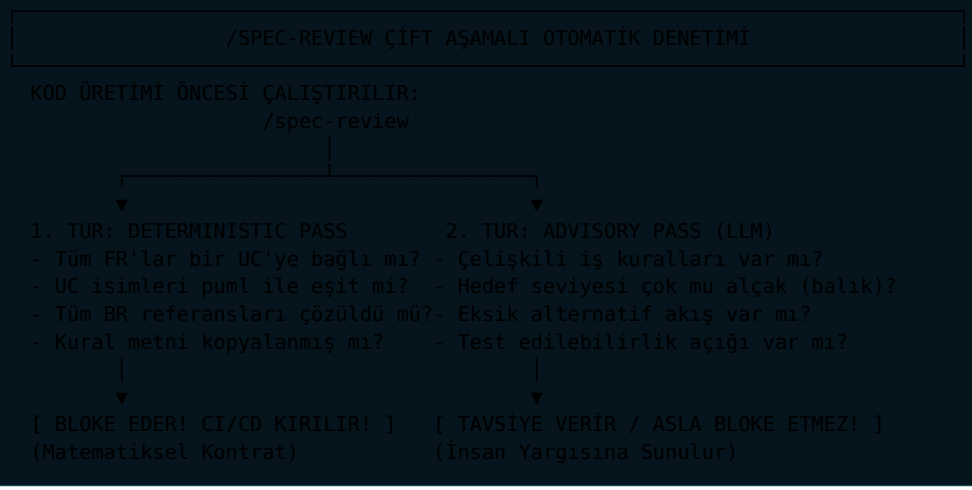
BÖLÜM: ESER ZINCIRI (ARTIFACT CHAIN) & PLANTUML MIMARISI | MODULE: THE ARTIFACT CHAIN & PLANTUML ARCHITECTURE

Otomatik Doğrulama: /spec-review Komutunun Deterministik ve Danışmanlık Turları

Automated Validation with /spec-review: Deterministic vs Advisory Passes

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Uçağa binmeden önce iki güvenlik kontrolünden geçersin: 1) Deterministik Geçiş (Turnike): Biletini makineye okutursun. Biletin sahte veya başkasının adıneysa turnike KÜT diye kapanır, geçemezsin (Bloke eder!). 2) Danışmanlık Geçişi (Hostes): Hostes sana gülümser ve 'Beyefendi üzerinizdeki ceket bu havada sizi biraz üşütebilir' der (Tavsiye!). Karar senindir, uçağa binmeni engellemez. `/spec-review` aracı da böyledir: Bozuk referans varsa kod üretimini durdurur; kural eksikliği hissederse kulağına fısıldar!

🇬🇧 ENGLISH (ELI5)

Airport security operates in two phases: 1) Deterministic Gate: The automated e-gate scans your passport barcode. If the checksum fails or the name mismatches, the metal barrier locks shut instantly (Blocking). 2) Advisory Gate: A flight attendant observes you boarding and notes: 'Sir, it is raining heavily at our destination; you may want a coat.' (Advisory). `/spec-review` implements this exact dual-pass protocol: syntax and reference mismatches halt the build; heuristic ambiguities are flagged for human review.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

/spec-review Mimarisinin Çift Aşamalı Güvencesi: Herhangi bir use case'i `/implement` komutuyla yapay zekaya kodlattırmadan önce, şartname zincirinin tutarlı olduğunu doğrulamak şarttır. AIUP bu görevi `/spec-review` becerisine (skill) verir. Bu denetim iki bağımsız turda çalışır: 1) Tur 1 - Deterministik Geçiş (Deterministic Pass): Tamamen deterministik kurallarla kod tabanını ve markdown dosyalarını tarar. Her FR-XXX en az bir UC'ye bağlı mı? Diyagramdaki her dosya diskte var mı? Atıfta bulunulan tüm `UC-001` `BR-002` referansları gerçekten mevcut mu? Kurallar kopyalanmış mı yoksa linklenmiş mi? Bu aşamada tek bir hata bulunursa CI derlemesi kırılır (Build Failure); AI kod üretemez. 2) Tur 2 - Danışmanlık Geçişi (Advisory Pass): LLM devreye girer ve anlamsal (semantic) analiz yapar. İş kuralları arasında mantıksal bir çelişki var mı? Hedef seviyesi deniz seviyesinden balık seviyesine düşmüş mü? Eksik bir hata akışı (alternative flow) kalmış mı? Bu bulgular asla CI'ı bloke etmez; şartname sahibine (Requirements Engineer) bir tavsiye raporu olarak sunulur. Bu iki aşamalı filtre, AI'a teslim edilen şartnamenin %100 kusursuz ve çelişkisiz olmasını garantiler.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Architecture of the /spec-review Verification Engine: Prior to dispatching specifications to `/implement`, the artifact chain must be audited for referential integrity. The `/spec-review` command executes in two decoupled passes: 1) Pass 1 (Deterministic Structural Audit): An algorithmic static analyzer parses markdown trees without LLM invocation. It verifies that every functional user story maps to at least one use case; that every use case node in `use_cases.puml` possesses a corresponding `.md` file; that all cross-spec citations (`UC-XXX` `BR-YYY`) resolve to actual declarations; and that no rule text was cloned. A failure here blocks the CI pipeline unconditionally. 2) Pass 2 (Advisory Semantic Audit): An LLM auditor inspects the narrative for cognitive defects: conflicting business logic, underwater goal altitudes, unaddressed failure paths, and untestable assertions. Findings from Pass 2 are advisory: they prompt human engineers to exercise judgment and never halt execution automatically.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: Deterministik /spec-review Doğrulayıcısının Çekirdek Mantığı
package com.maang.aiup.specreview;

import java.util.Set;

public class SpecReviewDeterministicEngine {
    // Kayıtlı gereksinimler ve bunları gerçekleştiren Use Case'ler
    public record VerificationReport(boolean passed, String errorMessage) {}

    public static VerificationReport verifyTraceability(
        Set<String> catalogFrIds,
        Set<String> realizedFrIds,
        Set<String> declaredRules,
        Set<String> referencedRules
    ) {
        // 1. Kural: Her FR en az bir Use Case tarafından gerçekleştirilmelidir!
        for (String fr : catalogFrIds) {
            if (!realizedFrIds.contains(fr)) {
                return new VerificationReport(false, "BLOKE EDİCİ HATA: " + fr + " hiçbir use case'e
bağlanmamış!");
            }
        }

        // 2. Kural: Referans verilen tüm BR'lar gerçekten var olmalıdır!
        for (String ref : referencedRules) {
            if (!declaredRules.contains(ref)) {
                return new VerificationReport(false, "BLOKE EDİCİ HATA: Çözülemeyen kural referansı ->
" + ref);
            }
        }

        return new VerificationReport(true, "✅ Deterministik denetim başarıyla tamamlandı.");
    }

    void main() {
        var frs = Set.of("FR-001", "FR-002");
        var realized = Set.of("FR-001", "FR-002");
        var declared = Set.of("UC-001 BR-001", "UC-002 BR-004");
        var refs = Set.of("UC-001 BR-001");

        var report = verifyTraceability(frs, realized, declared, refs);
        System.out.println(report.errorMessage());
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

/spec-review raporundaki 'Advisory' (Danışmanlık) uyarılarını görmezden gelip doğrudan koda geçmeyin. 'Bu akışta hata durumu eksik' uyarısı, yarın müşterinin sistemde yaşayacağı gerçek bir patlamanın habercisidir.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not dismiss Advisory findings from `/spec-review` casually. A warning regarding 'missing alternative flows' highlights an unhandled business condition that will manifest as unexpected behavior in production.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• /spec-review çift turludur: 1. Tur deterministik ve bloke edicidir, 2. Tur anlamsal tavsiyedir.

• Referansı çözülemeyen bir kural veya eksik bağlı bir FR varsa CI derlemesi kırılır.

• /spec-review asla dökümanları otomatik düzenlemez; sadece raporlar, karar insana aittir.

🎯 Key Architectural Takeaways

• /spec-review operates in two passes: Pass 1 is deterministic/blocking; Pass 2 is advisory.

• Dangling rule citations or orphaned requirements break the CI build deterministically.

• /spec-review reports findings without mutating files; decisions remain human-owned.